Appverse Technologies

WordPress Core and Environment Setup

Appverse Technologies — Internship Task 3

Submitted by:
Umm E Habiba

Organization:
Appverse Technologies

Project Title:
Complete Local WordPress Development Setup

Phase:
WordPress Foundations and Theme Development

Submission Date:
23 Oct 2026

1. Introduction and Objectives

This project focuses on creating a local WordPress development environment and extending WordPress through theme customization, plugins, hooks, filters, and user-role management.

The main objective is to establish a working WordPress installation on a local computer and document its technical configuration. The project also demonstrates how a child theme can be used to customize a website without modifying the parent theme directly.

The key objectives are:

- Configure a local WordPress development environment.
- Identify and document the WordPress database configuration.
- Install and activate five essential plugins.
- Develop and activate a custom child theme with enqueued styles.
- Implement five action hooks and three filter hooks.
- Create and test a custom user role with limited capabilities.
- Document the results and prepare the project for GitHub submission.

2. Environment and Database Setup

The project uses LocalWP to create and manage a local WordPress website. Visual Studio Code is used to edit theme files, while WordPress Admin is used to manage settings, plugins, themes, and users.

Environment configuration

- **Operating System:** Windows
- **Local Development Tool:** LocalWP
- **WordPress Version:** 7.1.3
- **PHP Version:** 8.2.29
- **Database Server:** MySQL 8.4.0
- **Web Server:** nginx
- **Code Editor:** Visual Studio Code

A new local website named Appverse-WordPress-Task3 was created. LocalWP provisioned the WordPress installation and database. The database was inspected to verify that the WordPress tables were available.

The site was configured with an appropriate title and the Post name permalink structure. Database credentials were kept private and were not included in the public repository.

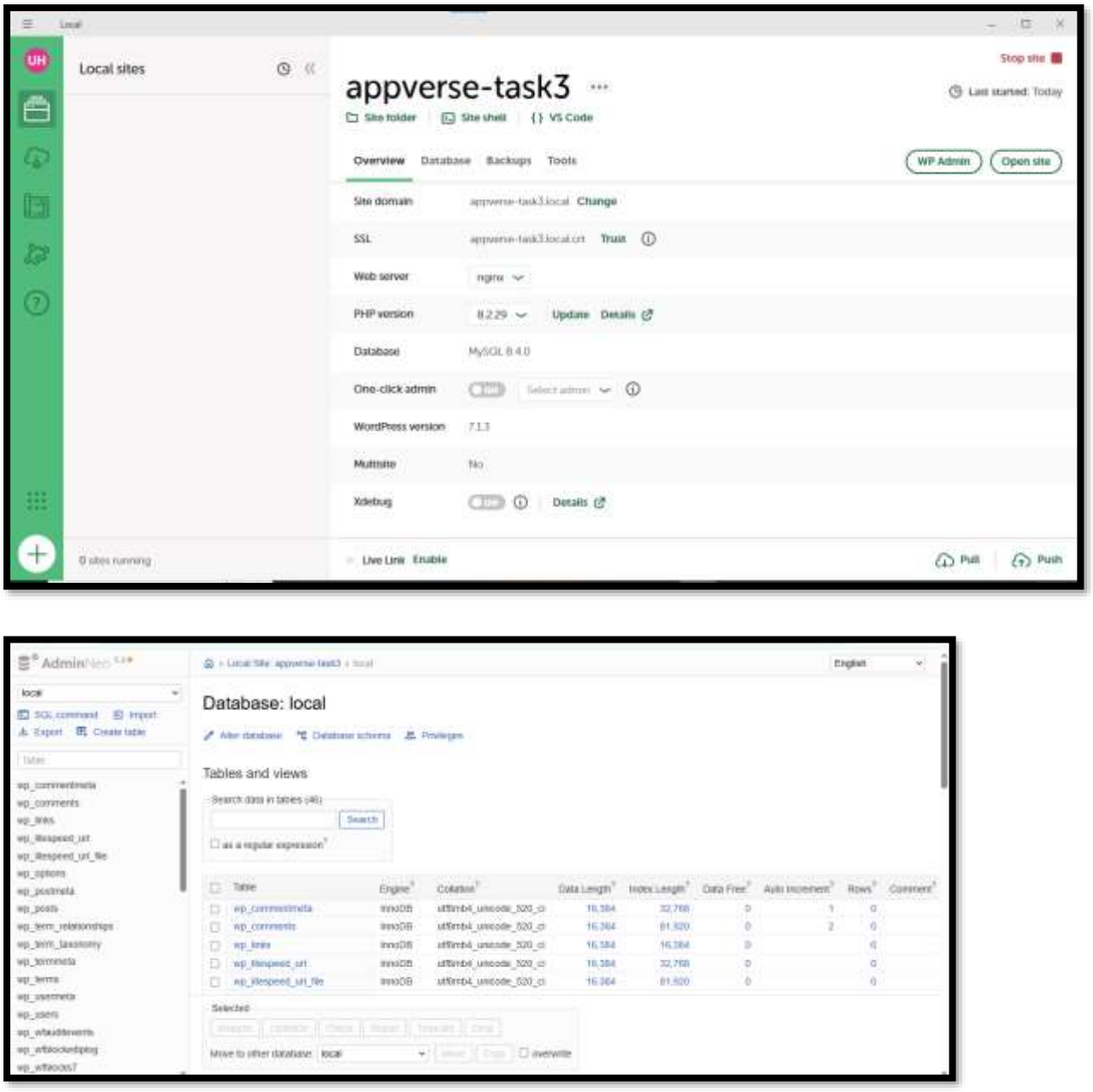


Figure 1. Local WordPress site created using LocalWP.

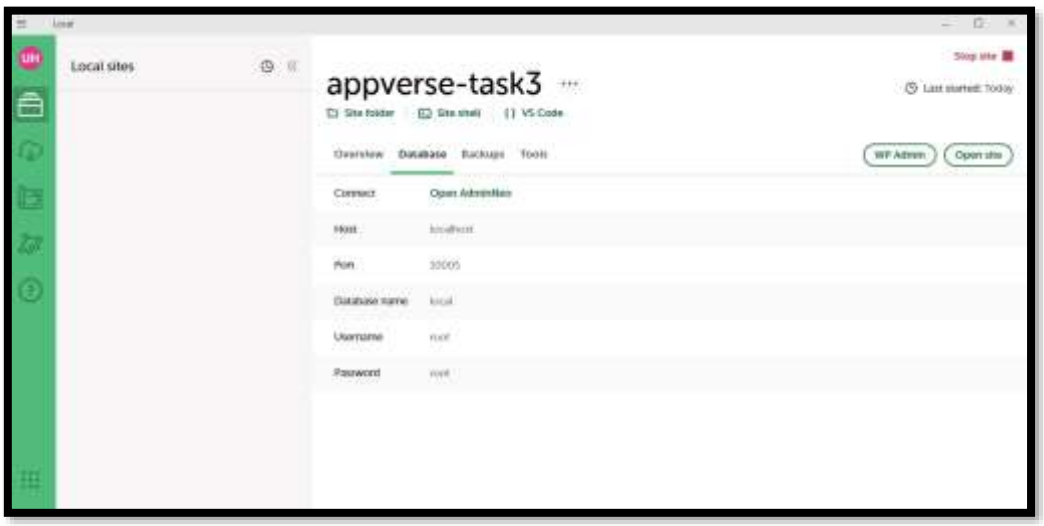


Figure 2. Local WordPress database and tables.

3. Plugin Installation and Theme Development

Five plugins were selected to demonstrate common WordPress functions:

1. **Yoast SEO:** Supports search engine optimization and metadata management.
2. **Wordfence Security:** Provides security monitoring and firewall functionality.
3. **UpdraftPlus:** Supports website backup and restoration.
4. **Contact Form 7:** Provides contact form functionality.
5. **LiteSpeed Cache:** Provides caching and performance optimization features, with full server-level caching dependent on compatible hosting.

The plugins were installed and activated through the WordPress dashboard.

A child theme named Appverse Child was created under the WordPress themes directory. The parent theme remained separate so that customizations could be maintained independently.

The child theme includes style.css for custom styling and functions.php for custom functionality. The stylesheet is loaded using WordPress enqueue functions. The custom styling changes heading colors, button appearance, and the site's font.

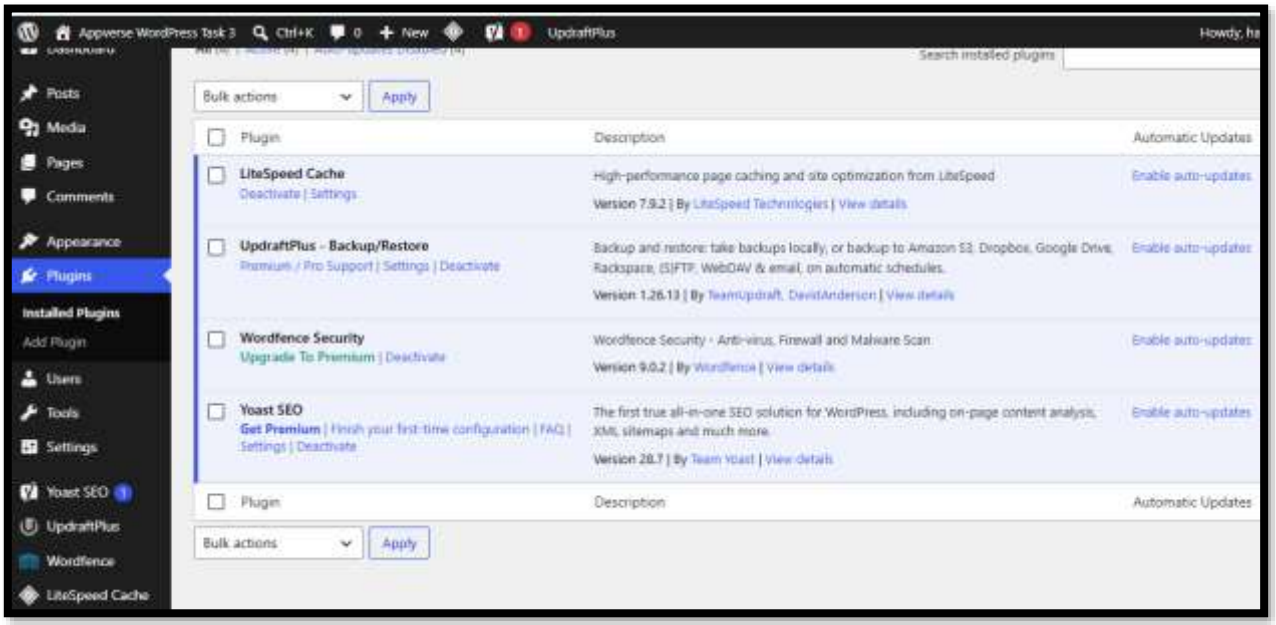


Figure 3. Five selected plugins installed and activated.

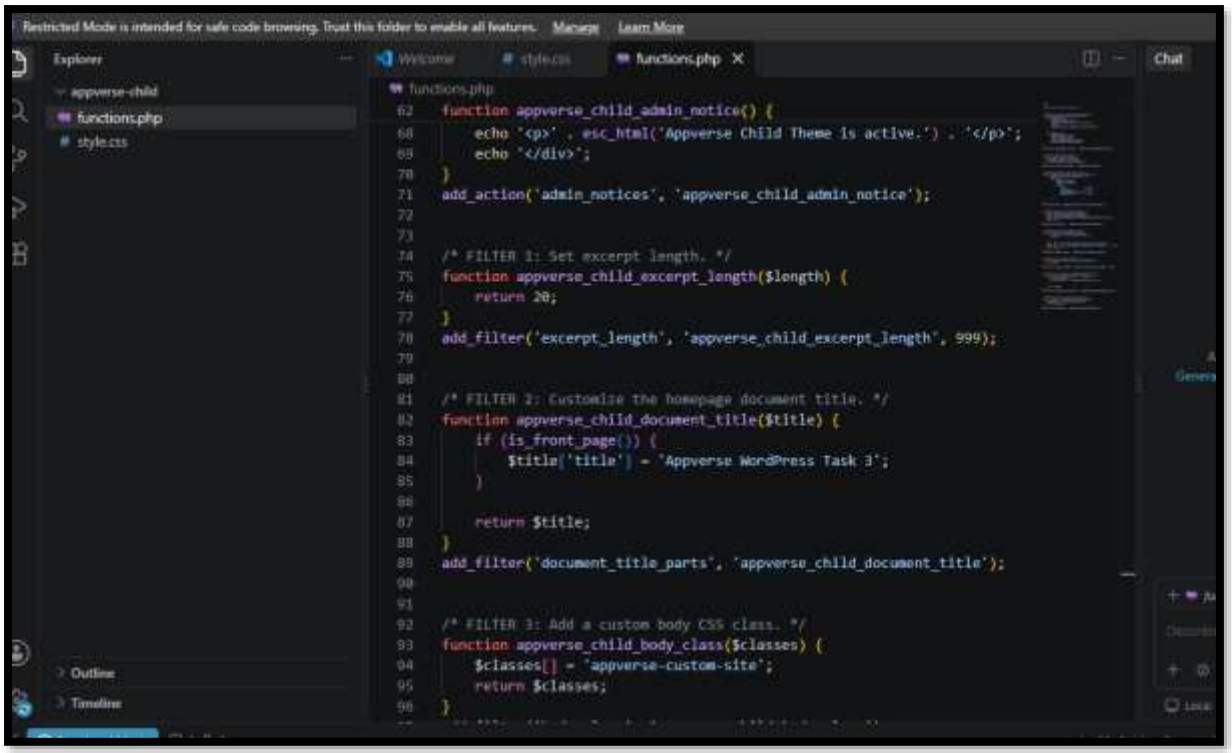


Figure 4. Child theme files in Visual Studio Code.

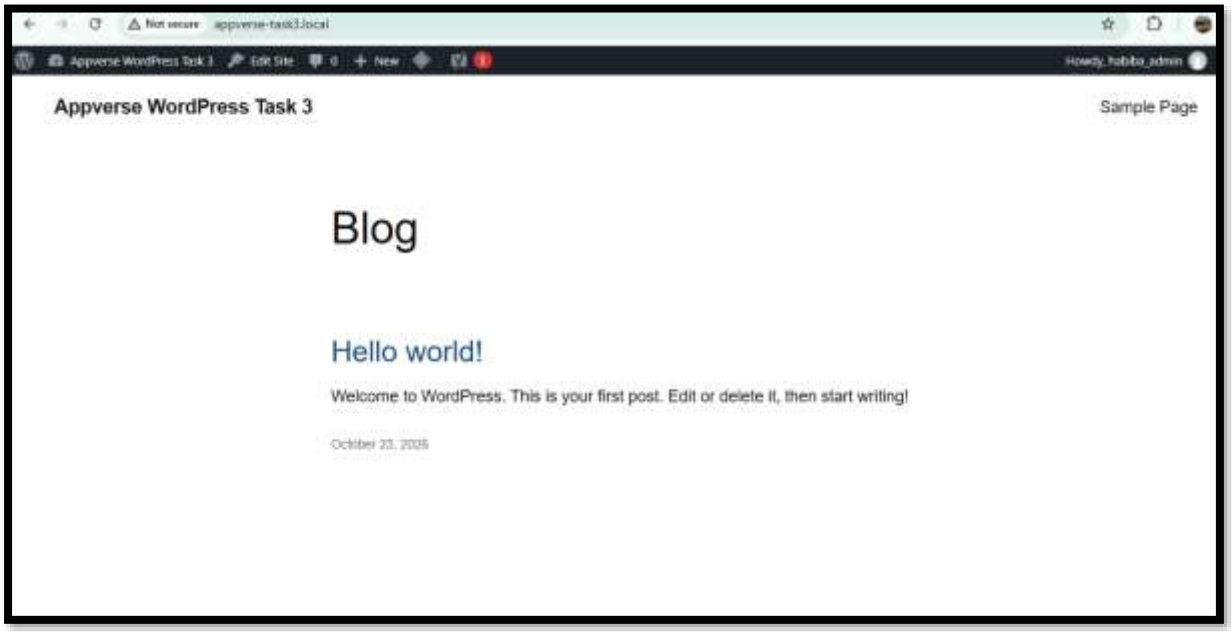


Figure 5. Website displaying the custom child theme styles.

4. Action Hooks, Filter Hooks, and User Roles

The child theme implements five action hooks:

- wp_enqueue_scripts loads the parent and child stylesheets.

- after_setup_theme enables theme support features.
- init registers the custom Content Assistant role.
- wp_footer displays a custom footer message.
- admin_notices displays a dashboard notification to administrators.

Three filter hooks are also implemented:

- excerpt_length changes the default excerpt length to 20 words.
- document_title_parts customizes the homepage document title.
- body_class adds the appverse-custom-site CSS class to the page body.

A custom role named **Content Assistant** was created with limited permissions. The role can read content, edit posts, edit its published posts, and upload files. It cannot publish posts or manage plugins and themes.

The role was tested using a separate user account. This demonstrates how WordPress capabilities can be used to restrict access according to a user's responsibilities.

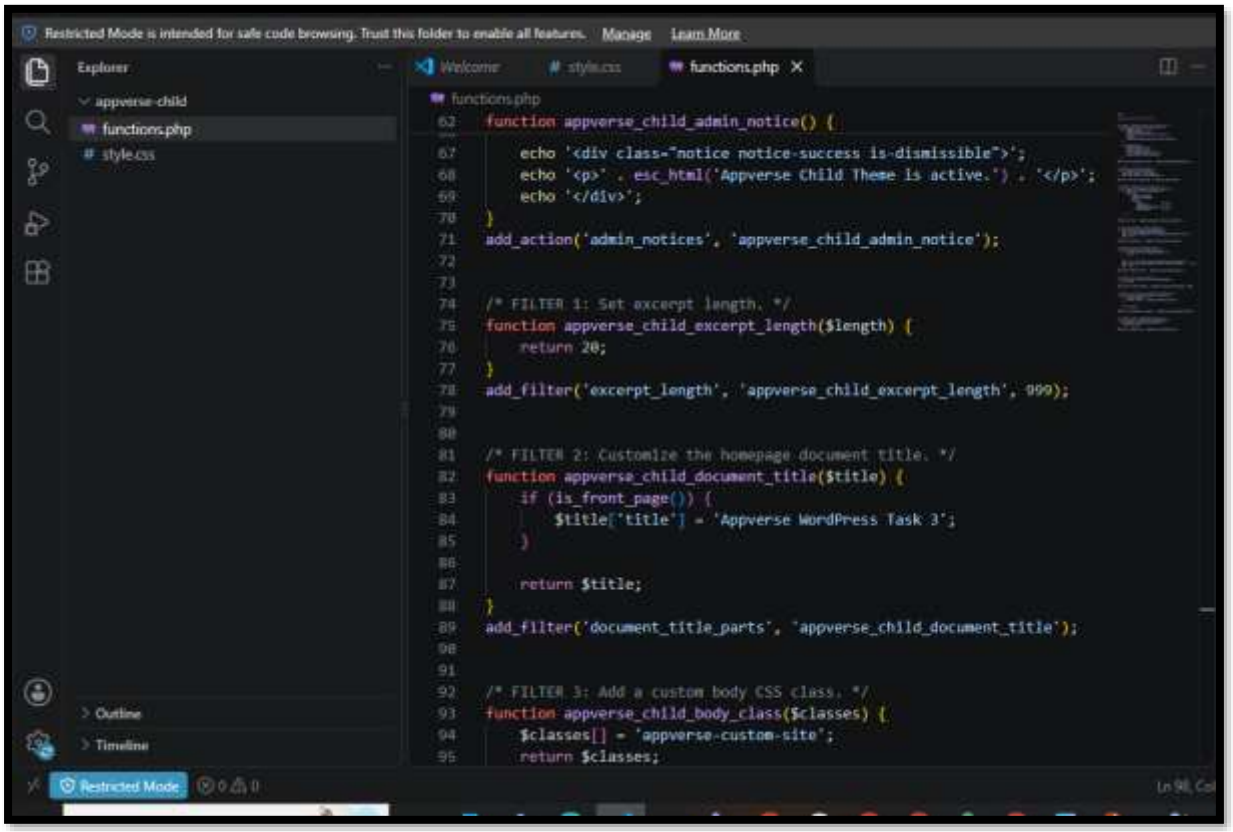


Figure 6. Five action hook implementations in functions.php.

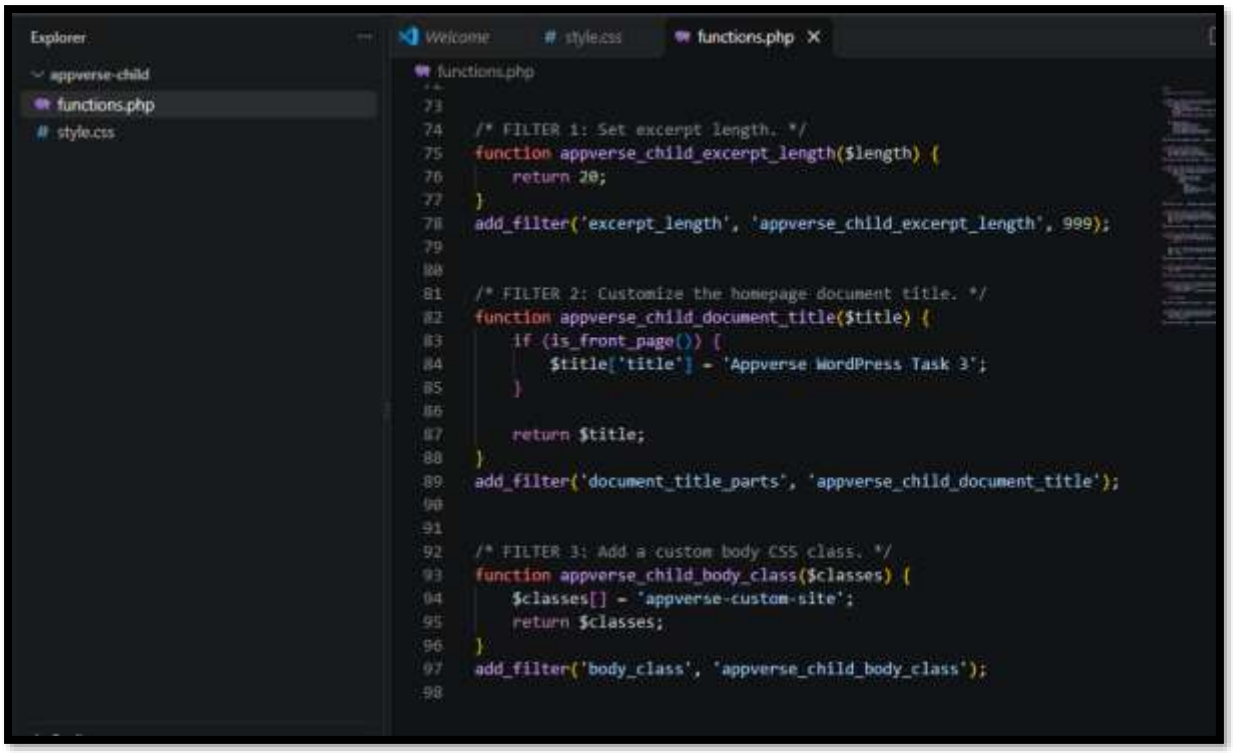


Figure 7. Three filter hook implementations in functions.php.

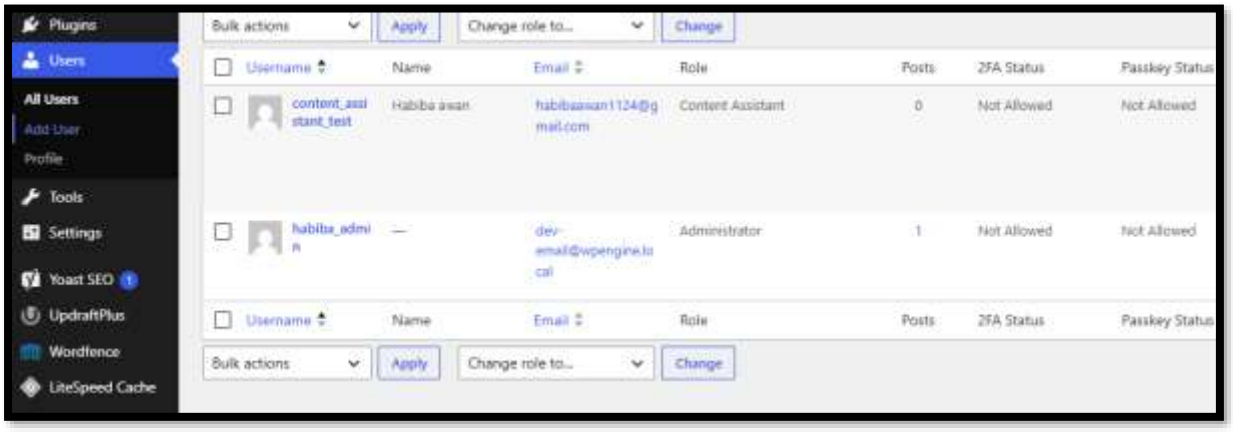


Figure 8. Testing the custom Content Assistant role.

5. Testing, Results, and Conclusion

5.1 Testing and Results

The project was tested through the WordPress dashboard and the public-facing website. Testing covered the local environment, database accessibility, plugin activation, child theme activation, custom styling, action and filter hooks, and user-role permissions.

The expected result is a functioning local WordPress development environment with documented configuration and customized theme functionality. The child theme provides custom styling, while the hooks and filters demonstrate how WordPress behavior can be

extended. The Content Assistant role provides an example of restricting user permissions based on assigned responsibilities.

The database configuration and installed plugins were also reviewed to confirm that the environment was prepared for local WordPress development.

5.2 Conclusion

This project provided practical experience in setting up a local WordPress development environment, reviewing database configuration, installing plugins, developing a child theme, and customizing WordPress using PHP hooks and filters.

The implementation of a custom user role also demonstrated the importance of managing user permissions and limiting access to administrative features. The project files, screenshots, and technical documentation provide a structured record of the development process.

In conclusion, the project establishes a foundation for local WordPress development and demonstrates how themes, plugins, hooks, filters, and user capabilities can be used to customize a WordPress website.

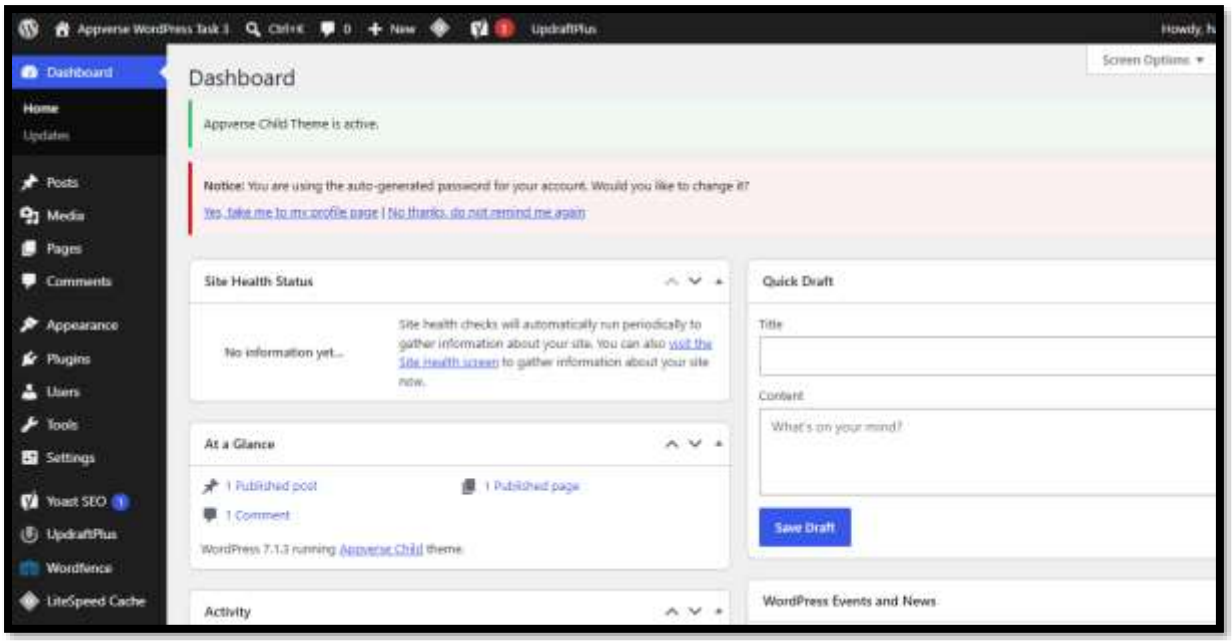


Figure: Final_WordPress_Dashboard.png

References

- WordPress.org. WordPress Documentation. https://wordpress.org/documentation/
- WordPress Developer Resources. Child Themes. https://developer.wordpress.org/themes/advanced-topics/child-themes/
- WordPress Developer Resources. Plugin API: Hooks. https://developer.wordpress.org/plugins/hooks/

- WordPress.org. Roles and Capabilities. https://wordpress.org/documentation/article/roles-and-capabilities/
- Local. Local WordPress Development. https://localwp.com/